



# Week 9 Lecture 4

|             |                            |
|-------------|----------------------------|
| ▼ Class     | BSCCS2003                  |
| 🕒 Created   | @November 2, 2021 12:03 PM |
| 🔗 Materials |                            |
| # Module #  | 51                         |
| ▼ Type      | Lecture                    |
| # Week #    | 9                          |

## HTTPS

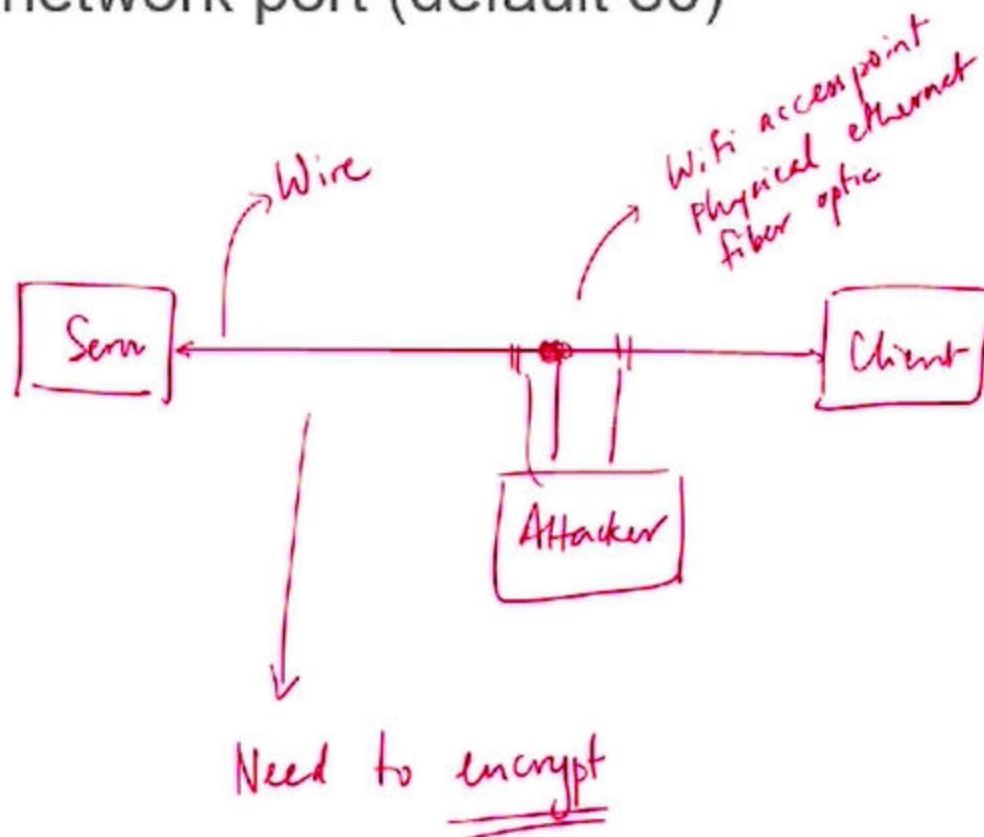
### Normal HTTP process

- Open connection to server on a fixed network port (default 80)
- Transmit HTTP request
- Receive HTTP response

Safety of transmitted data?

- Can be tapped
- Can be altered

## A network port (default 80)

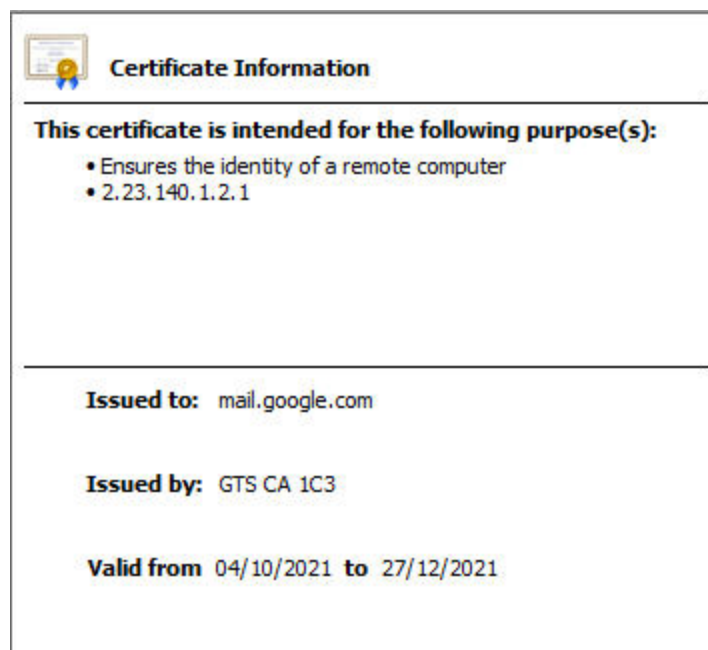


### Secure sockets

- Set up an "encrypted" channel between the client and the server
- How?
  - Need a shared secret → eg. long binary string → this is the "key"
  - XOR all the input data with key to generate new binary data
  - Attacker without the key cannot derive the actual data
- How to set up the shared secret?
  - Must assume anything on the wire can be tapped
  - What about pre-existing key?
  - Secure side channel → send a token by a post, SMS

### Types of security

- Channel (wire) security
  - Ensure that no one can tap the channel → most basic need for other auth mechanisms, etc.
- Server authentication
  - How do we know we are actually connection to mail.google.com and not some other server?
  - DNS hijacking is possible → redirect to another server
  - Server certificates
  - Common root of trust needed → someone who "vouches for" mail.google.com
- Client certificate
  - Rare but useful → server can require client certificate
  - Used especially in corporate intranets etc



## Chain of Trust

- Chain of trust
  - [mail.google.com](mailto:mail.google.com) issued certificate by
  - GTS CA1C3 issued certificate by
  - GTS Root R1
- GTS Root R1 certificate stored in Operating System or Browser
  - Do you trust your OS? Do you trust your browser?
    - interesting question, ngl
- From there on a secure (crypto) chain

## Potential problems

- Old browsers
  - Not updated with new chains of trust
- Stolen certificates at root of trust
  - Certificate revocation, invalidation possible
  - Need to ensure OS, browser can update their trust stores
- DNS hijacking
  - Give false IPs for server as well as entries along chain of trust
  - But certification in OS will fail against eventual root of trust

## Impact of HTTPS

- Security against wiretapping
- Better in public WiFi networks

Negative →

- Affects caching of resources (proxies cannot see content)
- Performance impact due to run-time encryption